

COMP3027 - Assignment 2

In this work, we present solutions to two problems employing dynamic programming techniques. The first task involves optimizing the selection of stormtroopers to minimize loyalty costs under interval constraints, while the second determines the feasibility of a path in a graph meeting specific time and scenery requirements.

Problem 1

In this problem, we aim to select a subset of stormtroopers such that the total loyalty cost is minimized, with the constraint that for any pair of stormtroopers with overlapping active intervals, at least one must be chosen. This corresponds to finding a minimum-cost vertex cover in an interval graph derived from the stormtroopers' active periods. We devise an efficient dynamic programming approach to achieve this.

a) Algorithm Description

We begin by arranging the stormtroopers in ascending order of their perishing times f_i , ensuring $f_1 \leq f_2 \leq \dots \leq f_n$. This ordering facilitates a sequential processing strategy essential to our solution.

For each stormtrooper j , we identify $p(j)$, defined as the largest index $i < j$ where $f_i < s_j$, indicating that stormtrooper i 's interval does not overlap with j 's. If no such i exists, we assign $p(j) = 0$. Given the sorted order, this computation is optimized using binary search.

Next, we define $S[j]$ as the maximum total cost of a non-overlapping subset of stormtroopers from indices 1 to j , representing an independent set in the interval graph. We initialize $S[0] = 0$, representing the empty set. For each j from 1 to n , we compute:

$$S[j] = \max(S[j-1], S[p(j)] + c_j)$$

Here, $S[j-1]$ denotes the maximum cost excluding stormtrooper j , while $S[p(j)] + c_j$ includes j combined with the optimal non-overlapping subset up to $p(j)$.

To reconstruct the optimal independent set, we traverse the S table backward from $j = n$. If $S[j] = S[j-1]$, stormtrooper j is not included, and we move to $j-1$. Otherwise, j is included, and we proceed to $p(j)$.

Finally, since the unselected stormtroopers form a vertex cover in the interval graph, we calculate its cost as $\sum_{i=1}^n c_i - S[n]$. Maximizing $S[n]$ ensures the vertex cover cost is minimized.

B) Proof of Correctness

We establish the correctness of our approach using mathematical induction.

For the base case, when $j = 0$, no stormtroopers are considered, so $S[0] = 0$, which accurately reflects the maximum cost of an empty independent set.

Assume that for all $k < j$, $S[k]$ correctly represents the maximum cost of an independent set over the first k stormtroopers. For j , consider two scenarios: excluding j yields $S[j - 1]$, which is optimal by the inductive hypothesis; including j requires combining its cost c_j with $S[p(j)]$, the optimal solution up to the latest non-overlapping predecessor. Since $p(j) < j$, $S[p(j)]$ is already computed and correct. Thus, taking the maximum of these options ensures $S[j]$ is the optimal cost for the first j stormtroopers.

The evaluation order from $j = 1$ to n guarantees that $S[j - 1]$ and $S[p(j)]$ are available when computing $S[j]$, satisfying dependencies. The backtracking process correctly identifies the subset by retracing the optimal choices. Since the vertex cover is the complement of the independent set, maximizing $S[n]$ minimizes the vertex cover cost, confirming the solution's validity.

C) Time Complexity Analysis

The algorithm's time complexity is analyzed as follows.

Sorting the n stormtroopers by perishing times requires $O(n \log n)$ time using an efficient algorithm such as mergesort, which is optimal for comparison-based sorting.

For each of the n stormtroopers, computing $p(j)$ involves a binary search over the sorted list of perishing times to find the largest $i < j$ where $f_i < s_j$. With n elements, each binary search takes $O(\log n)$ time, resulting in a total of $O(n \log n)$ for this step.

Filling the S table involves iterating from $j = 1$ to n . Each $S[j]$ computation uses $S[j - 1]$ and $S[p(j)]$, both previously calculated, and requires only a comparison and addition, taking $O(1)$ time per entry. Thus, this phase completes in $O(n)$ time.

Backtracking traverses the table from $j = n$ to $j = 1$, making a constant-time decision at each step based on whether $S[j] = S[j - 1]$. This single pass takes $O(n)$ time.

Summing these components, we have $O(n \log n)$ for sorting, $O(n \log n)$ for predecessor computation, $O(n)$ for dynamic programming, and $O(n)$ for backtracking. The dominant terms are $O(n \log n)$, yielding an overall time complexity of $O(n \log n)$.

Problem 2

This task requires determining whether an undirected graph contains a path from vertex u to vertex v with a total travel time at most T and a scenery score at least S . We develop a dynamic programming solution to efficiently explore all feasible paths.

A) Algorithm Description

We define $F[x, t]$ as the maximum scenery score achievable by reaching vertex x at exactly time t , where $x \in V$ and $t \in [0, T]$. If reaching x at time t is impossible, $F[x, t] = -\infty$.

Initialization sets $F[u, 0] = 0$, as we start at u with zero scenery, and $F[x, 0] = -\infty$ for all $x \neq u$, since no other vertex is reachable at time zero.

For each time step τ from 0 to $T - 1$, and for each edge $\{x, y\}$ with travel time t and scenery score o , we update the table in both directions due to the graph's undirected nature:

$$F[y, \tau + t] \leftarrow \max(F[y, \tau + t], F[x, \tau] + o)$$

$$F[x, \tau + t] \leftarrow \max(F[x, \tau + t], F[y, \tau] + o)$$

This ensures that all possible movements across edges are considered at each time increment.

We process τ sequentially, building the F table incrementally. After completing all updates, we examine $F[v, t]$ for $t = 0$ to T . If any $F[v, t] \geq S$, a qualifying path exists; otherwise, no such path is possible.

B) Proof of Correctness

We prove correctness through induction.

At time $t = 0$, $F[u, 0] = 0$ correctly indicates the starting position, and $F[x, 0] = -\infty$ for $x \neq u$ reflects that no travel has occurred, establishing the base case.

Assume that $F[x, s]$ accurately represents the maximum scenery score for reaching x at time s for all $s \leq \tau$. For time $\tau + t$, consider reaching vertex y via edge $\{x, y\}$. The score becomes $F[x, \tau] + o$, and we take the maximum over all such x adjacent to y . The symmetric update for x via y accounts for the undirected edges. Since $\tau < \tau + t$, the values at time τ are available, and the update ensures $F[y, \tau + t]$ captures the optimal score.

Processing τ from 0 to $T - 1$ ensures all prerequisite states are computed. The final check of $F[v, t] \geq S$ directly verifies the existence of a path meeting both constraints, as F encompasses all possible paths up to time T .

C) Time Complexity Analysis

The time complexity is detailed as follows.

Initializing the F table for time $t = 0$ involves setting $F[u, 0] = 0$ and $F[x, 0] = -\infty$ for $n - 1$ other vertices, taking $O(n)$ time.

The main computation loops over τ from 0 to $T - 1$, a total of T iterations. In each iteration, we process all m edges. For each edge $\{x, y\}$, we perform two updates: one from x to y and one from y to x . Each update involves accessing $F[x, \tau]$, adding o , comparing with $F[y, \tau + t]$, and possibly updating it, all in $O(1)$ time. Thus, processing all edges per τ takes $O(m)$ time.

Additionally, to manage the table efficiently, we may need to prepare the next time layer (e.g., ensuring $F[x, \tau + t]$ is initialized to $-\infty$ if not yet set). This can be done by initializing all $F[x, t]$ to $-\infty$ at the start for $t = 0$ to T , which takes $O(nT)$ time, or by handling it per step. Assuming a two-layer approach (current and next time), we reset or copy n entries per τ , costing $O(n)$ per iteration. However, the initial $O(nT)$ setup is more practical and subsumes this cost.

Total loop time is $T \cdot O(n + m) = O((n + m)T)$, combining edge updates and layer management. The final check examines $F[v, t]$ for $t = 0$ to T , taking $O(T)$ time, which is dominated by the loop.

The overall complexity is $O(nT)$ for setup plus $O((n + m)T)$ for updates, simplifying to $O((n + m)T)$, as T iterations each handle n vertices and m edges consistently with the problem's requirements.